

MSc Quantum Science and Technology

Master Thesis

Toward ML-Based State Discrimination on FPGA in Open-Source Quantum Control Electronics

Author: Ender Jose Barillas Rodriguez

Supervised by: Joel Pérez Díaz

NIUB: 23254663
Faculty of Physics, University of Barcelona
08028, Barcelona
July 2026

Acknowledgements

I would like to thank the team at Qilimanjaro Quantum Tech for their guidance and support throughout this project. Special thanks to Joel Pérez Díaz for supervising this work and to all colleagues who offered technical help and stimulating discussions. This thesis was carried out during an internship at Qilimanjaro from February to July 2026. **[TODO: Expand. Thank David A. for reviewing, and Fabio H. and the team for help with the board. Thank classmates. Thank Bruno and the program for letting me take a shot at joining a new field. Thank Arianna for helping me survive in Barcelona. Thank parents as well.]**

Abstract

Low-latency and high-fidelity qubit state measurement is essential for superconducting quantum computing, particularly for enabling mid-circuit measurements and real-time feedback. While commercial control hardware increasingly supports on-FPGA state discrimination, these platforms operate on closed firmware that precludes custom signal processing pipelines. Open-source FPGA frameworks offer an alternative path: full programmatic access to the readout chain, enabling researchers to integrate custom algorithms — including machine learning — directly into the measurement workflow.

This thesis documents the setup and characterisation of a QICK-based [1]readout system on a Xilinx ZCU216 RFSoC platform, covering FPGA configuration, remote operation, DDS-based waveform generation, and analogue front-end characterisation. The work establishes a functional signal chain capable of driving and reading out superconducting resonators, and examines the hardware constraints relevant to dispersive qubit readout, including balun response and filter rolloff across the relevant frequency bands.

Building on this infrastructure, we design and evaluate a machine-learning-assisted state discrimination pipeline using existing experimental single-shot readout data from a superconducting qubit device. We find that on integrated IQ data, linear discriminant analysis is already near-optimal — no MLP classifier improves upon it — consistent with the Gaussian blob structure of the IQ clouds. The only meaningful ML payoff comes from a lightweight 1D CNN trained on raw ADC traces, which recovers shots corrupted by mid-readout T1 relaxation that integrated methods cannot distinguish. This model is trained and evaluated entirely offline; real-time deployment within the QICK firmware was not achieved within the scope of this work. Instead, the thesis concludes by outlining the path toward such deployment: the data collection procedure required with the ZCU216 platform, the model adaptation and quantization constraints imposed by the programmable logic fabric, and the experimental methodology needed to assess whether deployment yields measurable improvements in classification speed, fidelity, or both.

Keywords: superconducting qubits, readout, FPGA, QICK, RFSoC, dispersive measurement, mid-circuit measurement, machine learning, state discrimination

Contents

1 Introduction

Quantum computing holds the promise of solving classically intractable problems in optimisation, simulation, and cryptography. Among the leading hardware platforms, superconducting qubits stand out for their relatively fast gate times, high coherence, and compatibility with existing microwave engineering infrastructure [2]. A central challenge that must be overcome on the road to fault-tolerant operation is *qubit state measurement*: reading out the quantum information stored in a qubit quickly, accurately, and with minimal back-action on unmeasured qubits.

1.1 Motivation

Classical error correction requires measurements that are not only high-fidelity but also *fast* relative to qubit coherence times, and — crucially for quantum error correction — it requires *mid-circuit measurements*: the ability to measure ancilla qubits in the middle of a quantum algorithm, use the result to apply corrections in real time, and continue the computation. This capability demands a control electronics stack capable of making a state discrimination decision on-chip in nanoseconds to microseconds, not the milliseconds typical of host-software processing.

Field-Programmable Gate Arrays (FPGAs) offer a compelling solution. Their deterministic, sub-microsecond timing enables real-time signal processing close to the qubit hardware. The Quantum Instrumentation Control Kit (QICK) [1] is an open-source FPGA firmware and software framework that turns Xilinx Radio-Frequency System-on-Chip (RFSoC) boards into complete qubit controllers, combining fast digital-to-analogue converters (DACs), analogue-to-digital converters (ADCs), and a tightly integrated soft-processor into a single compact platform.

1.2 Goals and Contributions

Despite the availability of QICK, assembling a *general-purpose* readout pipeline from an uncharacterised board to a working single-shot state discriminator requires integrating many layers: physical wiring, FPGA configuration, phase calibration, signal optimisation, and machine-learning post-processing. The lack of a unified reference implementation makes this process time-consuming and error-prone, especially for multi-qubit scenarios.

This thesis makes the following contributions:

1. **Full-stack integration** of the QICK framework on a Xilinx ZCU216 RFSoC board with an XM655 analogue front-end module, including remote operation via a persistent Pyro4 server.

2. **Phase-coherent readout calibration**: a systematic procedure for measuring and compensating the random DAC–ADC phase offset that arises at every board power-on, enabling reproducible IQ measurements across sessions.

3. **Readout data-path design**: a characterisation of the trade-offs between decimated and accumulated readout modes and a complete resonator spectroscopy workflow suitable for both single-qubit and multiplexed measurements.

4. **Real-time feedback primitives**: implementation and demonstration of conditional pulse logic on the tProcessor, establishing the active qubit reset and mid-circuit measurement infrastructure needed for dynamic quantum circuits.

5. **ML-assisted state discrimination**: evaluation of lightweight machine-learning classifiers for on-FPGA deployment, balancing fidelity, latency, and resource consumption.

1.3 Thesis Outline

Section 2 reviews the relevant theory of superconducting qubits, circuit quantum electrodynamics (cQED), and dispersive readout. Section 3 describes the hardware platform. Section 4 covers system integration and calibration. Section 5 presents the readout architecture and measurement workflows. Section 6 describes real-time feedback and mid-circuit measurement. Section 7 evaluates machine-learning-based state discrimination. Section 8 summarises the results and outlines future directions.

2 Theoretical Background

2.1 Superconducting Qubits

Superconducting qubits are anharmonic LC oscillators whose nonlinearity is provided by one or more Josephson junctions [2]. The Josephson junction acts as a nonlinear inductor with energy $E_J \cos\hat{\varphi}$, where $\hat{\varphi}$ is the superconducting phase across the junction and E_J is the Josephson energy. Combined with a shunt capacitance C (charging energy $E_C = e^2/2C$), the system forms an artificial atom with a discrete, anharmonic spectrum that can be addressed at the transition frequency ω_{01}.

Transmon. The transmon [3] operates deep in the regime $E_J \gg E_C$, making its energy levels insensitive to charge noise. Its transition frequency lies typically in the 4 GHz to 8 GHz range, and its anharmonicity $\alpha = \omega_{12} - \omega_{01}$ is of order $-E_C/\hbar$, typically a few hundred MHz.

Fluxonium. The fluxonium [4] replaces the transmon's large capacitor with a superinductance L (inductive energy $E_L = (\Phi_0/2\pi)^2/L$, where Φ_0 is the flux quantum), threading the loop with an external flux Φ_z. The resulting double-well potential can be tuned between a plasmon regime (near $\Phi_z = 0$) and a fluxon regime (near $\Phi_z = \Phi_0/2$). Fluxonium qubits offer long coherence times and are of particular interest because their readout landscape is richer: at half-integer flux bias the two wells become degenerate and the qubit transition frequency passes through zero, enabling so-called *Which-Well Readout* (WWR) [5].

2.2 Circuit Quantum Electrodynamics

In the circuit QED (cQED) architecture [6], a qubit is coupled dispersively to a microwave resonator. The resonator acts both as a readout bus and as a filter that protects the qubit from radiative decay into the measurement chain. The full Hamiltonian (setting $\hbar = 1$) is

$$\hat{H} = \hat{H}_{\text{qubit}} + \omega_r \hat{a}^\dagger \hat{a} + g\, \hat{n}_q (\hat{a} + \hat{a}^\dagger), \tag{1}$$

where ω_r is the resonator frequency, $\hat{a}$ ($\hat{a}^\dagger$) is the resonator annihilation (creation) operator, g is the qubit–resonator coupling strength, and $\hat{n}_q$ is the relevant qubit operator (charge for capacitive coupling).

2.3 Dispersive Readout

When $|g_{ij}| \ll |\omega_{ij} - \omega_r|$ for all relevant qubit transitions $i \leftrightarrow j$, second-order perturbation theory in g yields a qubit-state-dependent shift of the resonator frequency [5]:

$$\hat{H}_{\text{disp}} = \left(\omega_r + \sum_i \chi_i \left|i\right\rangle \left\langle i\right|\right) \hat{a}^\dagger \hat{a}, \quad \chi_i = \sum_{j \neq i} \frac{|g_{ij}|^2}{\omega_{ij} - \omega_r} - \frac{|g_{ji}|^2}{\omega_{ji} - \omega_r}, \tag{2}$$

where $g_{ij} = \langle i | g\, \hat{n}_q | j \rangle$ and $\omega_{ij} = \omega_j - \omega_i$. The textbook two-level dispersive shift $\chi = \chi_1 - \chi_0$ follows from truncating to the computational subspace, but for fluxonium-class qubits the straddling regime (where $|g_{12}| \gg |g_{01}|$) makes this truncation unsafe; higher levels must be included [3].

In practice, one probes the resonator at its bare frequency ω_r and observes a state-dependent phase shift of the transmitted or reflected signal. For a single-qubit system, the resonator sits at $\omega_r + \chi_0$ when the qubit is in $|0\rangle$ and at $\omega_r + \chi_1$ when in $|1\rangle$, resulting in an IQ-plane separation that grows with the dimensionless cavity pull $2|\chi|/\kappa$, where κ is the resonator linewidth [6].

Which-Well Readout. For a double-well fluxonium biased off degeneracy, the eigenstates are approximately localised in the left ($|L\rangle$) or right ($|R\rangle$) well and carry definite persistent-current sign. Because the two wells host different local spectra, the dispersive shift evaluated on the ground state of each well differs, and the half-splitting $\chi_{\text{WWR}} = \frac{1}{2}(\chi_R - \chi_L)$ sets the WWR contrast [5].

2.4 Direct Digital Synthesis

Direct Digital Synthesis (DDS) generates analogue waveforms by accumulating a phase register at a fixed clock rate and mapping the instantaneous phase value to a digital amplitude via a look-up table. A DDS numerically controlled oscillator (NCO) produces a tone at frequency

$$f_{\text{out}} = \frac{f_{\text{clk}} \cdot \Delta\phi}{2^N}, \tag{3}$$

where f_{clk} is the system clock, $\Delta\phi$ is the phase increment per clock cycle, and N is the bit width of the phase accumulator. DDS avoids the spur and image problems associated with analogue IQ mixers because all imperfections are digital and predictable. In QICK, the tProcessor drives DDS engines on both the DAC (transmit) and ADC (receive) sides, enabling phase-locked signal generation and coherent demodulation [1].

2.5 State Discrimination

Following dispersive readout, the measured IQ quadrature amplitudes (I, Q) form two clusters in the complex plane — one for $|0\rangle$ and one for $|1\rangle$. Single-shot state discrimination assigns each measurement to one of these clusters. The simplest approach is a linear threshold: a hyperplane in the IQ plane that minimises the total misclassification probability, which for Gaussian clusters of equal covariance is the perpendicular bisector of the line connecting the centroids [7]. The single-shot readout fidelity is

$$\mathcal{F} = 1 - \frac{1}{2}\left(p(1|0) + p(0|1)\right), \tag{4}$$

where $p(1|0)$ is the probability of assigning $|1\rangle$ given that the qubit was prepared in $|0\rangle$, and vice versa. More sophisticated classifiers — quadratic discriminant analysis, neural

networks — can improve fidelity when the clusters are non-Gaussian or when multi-level leakage is present.

3 Hardware Platform

3.1 Xilinx ZCU216 RFSoC

The Xilinx Zynq UltraScale+ ZCU216 evaluation board is built around an RFSoC device that integrates, on a single chip, a quad-core ARM Cortex-A53 processor, an FPGA fabric, and 16 RF-DAC and 16 RF-ADC channels with sampling rates up to 9.85 GS/s and 2.5 GS/s, respectively. Operating at 12-bit and 14-bit resolution, the converters can directly synthesise and capture signals in the 0 GHz to 6 GHz range without an external frequency up-conversion stage, provided the analogue signal chain is suitably conditioned.

The board runs the PYNQ Linux distribution, which exposes the FPGA fabric and the converters through Python libraries, making it straightforward to implement and iterate on firmware/software stacks. Table 1 summarises the key specifications relevant to this thesis.

Table 1: Selected ZCU216 specifications relevant to qubit readout.

Parameter	Value	Note
DAC channels	16	Differential GSPS
DAC rate	up to 9.85 GS/s	Per channel
DAC resolution	14 bit	
ADC channels	16	Differential GSPS
ADC rate	up to 2.5 GS/s	Per channel
ADC resolution	12 bit	
FPGA fabric	UltraScale+	930k LUT
Processor	4× Cortex-A53	1.5 GHz
Operating system	PYNQ Linux	

3.2 QICK Software and Firmware

The Quantum Instrumentation Control Kit (QICK) [1] is an open-source framework originally developed at Fermilab that turns an RFSoC board into a self-contained qubit controller. It consists of two layers:

Firmware. A set of Vivado IP blocks synthesised and placed onto the FPGA fabric, comprising:

- **Signal generators**: DDS-based waveform engines that drive the DACs with arbitrary envelopes, constant tones, or flat-top pulses. Generator 6 (physical port 2_231 on the ZCU216) is used as the primary readout generator throughout this work.
- **Readout blocks**: ADC-side DDS demodulators that down-convert the received signal to baseband. Each readout block provides both a decimated output (time-domain I/Q waveform) and an accumulation buffer (averaged I/Q pair). Readout channel 0 (ADC port 0_226) corresponds to the first ADC pair on the XM655 breakout.

- **tProcessor-64**: a soft-processor that orchestrates pulse timing with nanosecond precision. It executes a custom instruction set including direct memory writes (`memri`), register arithmetic (`mathi`, `regwi`), conditional branches (`condj`), and pulse triggers. The ZCU216 uses the 64-bit variant of the tProcessor, which extends the original 32-bit design with wider arithmetic and richer branching.

Software. A Python library (the `qick` package) that runs on the ARM core and exposes high-level classes for writing and running experiments:

- `QickSoc`: instantiates the full hardware stack, loads the bitstream, and provides access to the converters and tProcessor.
- `QickProgram` / `AveragerProgram` / `RAveragerProgram`: abstractions for writing measurement sequences at increasing levels of automation. `RAveragerProgram` enables FPGA-internal parameter sweeps via the tProcessor's `update()` hook, avoiding Python overhead between experiment points.
- Remote operation: a persistent Pyro4 server running on the board makes it possible to control the board from a laptop over a standard network connection, without reinitialising the FPGA between notebook sessions. This design also preserves the phase calibration across reconnects.

3.3 XM655 Analogue Front-End Module

The ZCU216's RF converters use differential signalling on High-Density (HD) headers (JHC3 and JHC5 on the board). Most laboratory equipment, however, uses single-ended SMA connectors. The XM655 module is Xilinx's daughter-card solution: it provides BALUN (balanced-to-unbalanced) transformers that convert the differential DAC outputs and ADC inputs to/from single-ended SMA ports, along with optional filtering.

Key constraints introduced by the XM655 in this work:

- The BALUNs on the XM655 cover the frequency range 10 MHz to 1000 MHz. Resonator frequencies above 1 GHz require connections directly on the HD header pins using pigtail cables and separate discrete BALUNs.
- Only a limited number of BALUN-equipped SMA ports are available in the 3 GHz to 6 GHz band, constraining the number of simultaneous readout tones.

The loopback path used for calibration experiments consists of: DAC port 2_231 (JHC3) → HD2-SMA pigtail → BALUN → SMA cable → BALUN → HD2-SMA pigtail → ADC port 0_226 (JHC5).

Figure 1 shows the high-level signal flow from the tProcessor to the qubit chip and back.

4 System Integration

4.1 FPGA Configuration and Board Bring-up

The ZCU216 boots the PYNQ Linux image from an SD card. After boot, the QICK bitstream is loaded onto the FPGA fabric by instantiating a `QickSoc` object in Python, which calls the PYNQ overlay loader and configures all IP blocks. Because the RFSoC

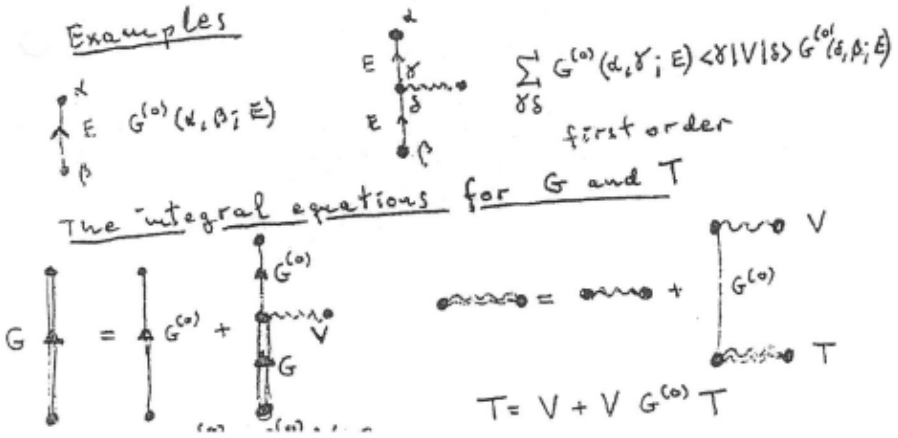

Figure 1: System topology placeholder. The tProcessor generates pulse sequences that drive the DAC signal generators; the analogue signal passes through the XM655 front-end to the qubit chip. The reflected/transmitted readout tone is routed through the cryogenic chain back to the ADC via the XM655. **[TODO: Replace with actual system diagram.]**

converters are controlled through AXI register maps rather than external chip-select lines, initialisation is entirely software-driven.

The board is accessed over SSH. To avoid re-initialising the hardware for every new notebook session, a persistent Pyro4 object server is launched once on the board and kept running:

```
# On the board (runs once per power cycle)
import Pyro4, qick
soc = qick.QickSoc()
daemon = Pyro4.Daemon()
uri = daemon.register(soc, "qick.soc")
Pyro4.locateNS().register("qick.soc", uri)
daemon.requestLoop()
```

From a remote laptop:

```
import Pyro4
soc = Pyro4.Proxy("PYRONAME:qick.soc")
soccfg = soc  # lightweight config object
```

This design preserves the phase calibration state (see Section 4.3) across reconnects and makes experiment iteration fast.

4.2 Channel Mapping

The QICK firmware assigns each physical DAC output to a numbered *generator channel* and each physical ADC input to a numbered *readout channel*. On the ZCU216 with the default QICK bitstream, the mapping relevant to this work is shown in Table 2.

4.3 Phase Calibration

Each time the ZCU216 is powered on or the FPGA bitstream is reloaded, the DDS oscillators in the DAC and ADC blocks boot with a random mutual phase offset ϕ. Without compensation, IQ measurements rotate unpredictably between sessions, making it impossible to compare results across power cycles.

Table 2: QICK channel mapping on the ZCU216.

QICK Channel	Physical Port	HD Header	Function
Generator 6	DAC 2_231	JHC3	Readout / control drive
Readout 0	ADC 0_226	JHC5	IQ acquisition

Phase model. A loopback tone from DAC 2_231 to ADC 0_226 measures ϕ at a given frequency. Because the two oscillators are effectively separated by a fixed time delay τ (the round-trip propagation delay in the loopback cable), the phase offset scales linearly with frequency:

$$\phi(f) = \phi_0 - 360^\circ \cdot \tau \cdot f, \tag{5}$$

where ϕ_0 is a constant offset and $\tau \approx 251\,\mu s$ in the as-assembled loopback (dominated by the cable length, not the on-chip delay).

Calibration procedure. The phase calibration sweeps a range of frequencies f_i near the target readout frequency, records the averaged IQ phasor at each point, and extracts $\phi_i = \arg\langle I + jQ\rangle$. The pair (ϕ_0, τ) is then fitted using `scipy.optimize.least_squares`, taking care to handle the $0^\circ/360^\circ$ phase wrap correctly. The fit is considered reliable when the signal-to-noise ratio satisfies $|\text{mag}| \gg \text{RMS}$ (e.g. $637 \gg 1.8$ ADU in a typical calibration run).

The resulting `res_phase` value:

$$\phi_{\text{res}} = \phi_0 - 360^\circ \cdot \tau \cdot f_{\text{res}} \tag{6}$$

is stored in the experiment configuration dictionary and passed to the readout pulse via `soccfg.deg2reg(phi_res)`, which converts degrees to the integer register format expected by the tProcessor. This calibration must be re-run after every board reboot or bitstream reload, but remains valid throughout a measurement session.

Measurement cadence. A summary of the recommended start-of-session checklist is:

1. Power on and wait for PYNQ to boot (≈60 s).
2. Connect the loopback (DAC 2_231 → BALUN → SMA → BALUN → ADC 0_226) and run the phase calibration notebook.
3. Identify the correct `adc_trig_offset` using decimated readout (see Section 5.2).
4. Switch to accumulated readout for all data-taking.
5. At session end, call `soc.reset_gens()` to halt all running generators.

5 Readout Architecture

5.1 DDS-Based Signal Generation

In a standard heterodyne readout chain, a local oscillator (LO) at frequency f_{LO} is mixed with a baseband envelope to up-convert the readout tone to the resonator frequency f_r. This analogue mixing process introduces image products and LO leakage that degrade fidelity, requiring careful IQ imbalance correction.

QICK avoids analogue mixing entirely. Both the readout drive and the demodulation reference are produced by the on-chip DDS engines, which operate at the direct output frequency of the RFSoC converters (up to several GHz). The waveform generated for qubit readout has the form

$$s(t) = A \cdot \mathrm{env}(t) \cdot \cos(2\pi f_r t + \phi_{\mathrm{res}}), \tag{7}$$

where $\mathrm{env}(t)$ is the pulse envelope (constant, Gaussian, DRAG, etc.), f_r is programmed directly into the DDS frequency register, and ϕ_{res} is the phase correction from calibration. The only noise sources are digital: quantisation, clock jitter, and DDS phase truncation spurs — all of which are deterministic and can be mitigated by appropriate filtering and sample-rate choice.

5.2 Decimated vs. Accumulated Readout

After the ADC samples the incoming signal, the QICK readout block demodulates it by multiplying with the NCO reference tone and low-pass filtering. Two output modes are available:

Decimated readout. Stores the full time-domain I/Q waveform over the readout window (e.g. 1000 samples). This is essential for setup: it reveals when the readout pulse arrives relative to the trigger (`adc_trig_offset`), whether the integration window is correctly positioned, and whether the signal shape is as expected. The single-shot noise is $\sim$1.9 ADU RMS.

Accumulated readout. The FPGA sums all ADC samples in the readout window into a single (I, Q) pair in hardware. With N_{reps} repetitions, the noise falls as $1/\sqrt{N_{\mathrm{reps}}}$:

$$\sigma_{\mathrm{accum}} = \frac{\sigma_{\mathrm{single}}}{\sqrt{N_{\mathrm{reps}}}} \approx \frac{1.9\,\mathrm{ADU}}{\sqrt{N_{\mathrm{reps}}}}. \tag{8}$$

At $N_{\mathrm{reps}} = 1000$ this yields $\sigma \approx 0.14\,\mathrm{ADU}$ — a $31.6\times$ noise reduction. All experimental data in this thesis use accumulated readout once the timing and phase have been verified in decimated mode.

The practical workflow is therefore: (1) use `acquire_decimated()` to find the correct `adc_trig_offset` and verify signal quality; (2) switch to `acquire()` (accumulated) for all data-taking.

5.3 Resonator Spectroscopy

The resonator bare frequency ω_r and linewidth κ are found by sweeping the probe frequency around the estimated resonator frequency and measuring the transmitted or reflected IQ amplitude. In QICK, there are two sweep strategies:

Fast sweep (`RAveragerProgram`). The tProcessor increments the DAC DDS frequency register at each experiment step via the `update()` method. This approach is fast because the FPGA handles all steps with no Python round-trips. However, the ADC NCO frequency is set via the ARM AXI bus and cannot be updated by the tProcessor. For resonator spectroscopy, both the DAC and ADC frequencies must track together; this is not achievable with the fast method.

Slow sweep (Python outer loop). A `for` loop in Python updates both the DAC generator frequency and the ADC readout frequency at each step. The per-step overhead is a few milliseconds (dominated by network round-trips in remote mode), but this approach can sweep *any* parameter, including the ADC NCO. This is the correct approach for resonator spectroscopy.

In practice:

```
freqs = np.linspace(f_start, f_stop, n_points)
amps  = []
for f in freqs:
    cfg["pulse_freq"] = f
    prog = AveragerProgram(soccfg, cfg)
    iq   = prog.acquire(soc, load_pulses=True, progress=False)
    amps.append(np.abs(iq[0][0][0] + 1j*iq[0][0][1]))
```

The resonator frequency is identified as the dip (reflection) or peak (transmission) in the swept amplitude response.

5.4 Multiplexed Readout

For multi-qubit systems, simultaneous readout of several qubits on the same physical line requires *multiplexed readout*: a broadband probe signal containing multiple tones at the individual resonator frequencies $\{f_{r,k}\}$. The QICK framework supports this through a *Polyphase Filter Bank* (PFB) readout mode.

A PFB is a bank of digital bandpass filters implemented as a single, efficient DSP operation (polyphase decomposition of a prototype filter followed by an FFT). It takes the broadband ADC stream and simultaneously demodulates N equally-spaced frequency channels, analogous to a digital spectrum analyser. Advantages over the single-tone DDS approach include:

- No need to retune the ADC NCO between qubits.
- Computational cost scales as $O(N \log N)$ rather than $O(N)$ independent demodulators.
- Sidesteps the ADC-frequency-sweep limitation: each PFB output channel is a fixed digital filter, not a tunable DDS.

[TODO: Describe PFB implementation status and preliminary results, if available.]

6 Real-Time Feedback and Mid-Circuit Measurement

A key motivation for FPGA-based control is the ability to execute *real-time* conditional logic: perform a measurement, make a decision, and apply a corrective pulse — all on the FPGA, without a round-trip to the host CPU. This section describes the tProcessor-64 instruction set, demonstrates the conditional branching primitive, and outlines the active qubit reset protocol.

6.1 tProcessor Architecture

The tProcessor-64 is a soft-processor synthesised in the FPGA fabric. It executes a custom instruction set tailored to pulse generation and readout sequencing. Key features include:

- **Register file**: 32 general-purpose 64-bit registers per channel page, plus cross-page registers for inter-channel synchronisation.
- **Pulse triggers**: dedicated instructions that fire DAC signal generators or arm ADC acquisition windows.
- **Timing instructions**: `synci` (advance tProcessor time without touching channel timelines), `sync_all` (synchronise all channels), `wait_all` (stall until all readout windows close).
- **Conditional branch**: `condj page, reg_a, op, reg_b, label` jumps to `label` if `reg_a op reg_b` is true.

All branching targets are resolved at *compile time* by the Python assembler: Python's sequential execution of `body()` builds an instruction list, `label()` records the current position as a named address, and the assembler backpatches jump targets before loading the program onto the tProcessor. By the time the program runs, only integer addresses exist; there is no runtime symbol table.

6.2 Conditional Logic Demonstration

The QICK Demo 3 notebook demonstrates the core conditional-branch primitive:

```
def body(self):
    cfg = self.cfg
    self.trigger(adcs=[cfg["ro_ch"]],
                 adc_trig_offset=cfg["adc_trig_offset"])
    # condj: jump to SKIP_PULSE if number < threshold
    self.condj(0, 2, '<', 1, 'SKIP_PULSE')
    self.pulse(ch=cfg["res_ch"])          # skipped if condition true
    self.label('SKIP_PULSE')
    self.wait_all()
    self.sync_all(self.us2cycles(cfg["relax_delay"]))
```

Registers 1 and 2 are pre-loaded with the threshold and the test value via `regwi`. When `number=100, threshold=50`: the condition `100 < 50` is false, the pulse fires, and the ADC captures a loopback signal. When `number=10, threshold=50`: the condition is true, the pulse is skipped, and the ADC trace is flat noise.

Timing subtlety. Inside a `body()` with an open ADC acquisition window, `synci` must be used to advance time between pulses, not `sync_all`. `sync_all` advances all channel timelines to the furthest point, which is the end of the ADC window; subsequent pulses would then be scheduled after the window has closed. `synci` advances only the tProcessor's internal counter, leaving the ADC timeline undisturbed.

6.3 π Pulse Calibration

A π pulse rotates the qubit state by 180° on the Bloch sphere: $|0\rangle \to |1\rangle$ and $|1\rangle \to |0\rangle$. It is a fundamental primitive for qubit initialisation (active reset) and single-qubit gates.

The π pulse amplitude is calibrated with a *Rabi experiment*: the qubit drive pulse amplitude is swept from zero, and the readout signal oscillates between the $|0\rangle$ and $|1\rangle$ IQ clusters as the qubit is driven through successive half-turns on the Bloch sphere. The first amplitude at which the signal crosses from the $|0\rangle$ cluster to the $|1\rangle$ cluster corresponds to the π pulse. The Rabi sequence is:

1. Prepare qubit in $|0\rangle$ (wait $\gg T_1$).
2. Apply qubit drive pulse at frequency ω_{01} with variable amplitude A and fixed duration.
3. Read out via the resonator.

Note that the qubit drive tone is at ω_{01} (the qubit transition frequency, typically 5 GHz to 7 GHz for fluxonium), which is distinct from the resonator frequency ω_r used for readout. Two separate generator channels are therefore required: one for readout, one for qubit control.

6.4 Active Qubit Reset

Active reset is the process of rapidly returning the qubit to $|0\rangle$ by measuring its state and applying a corrective π pulse only if the qubit is found in $|1\rangle$:

1. **Dispersive readout**: fire the readout tone and acquire the accumulated IQ result.
2. **Thresholding**: the readout firmware compares the IQ result against a pre-calibrated threshold and writes a binary decision (0 or 1) into a tProcessor register.
3. **Conditional π pulse**: `condj` checks the decision register. If the qubit is in $|1\rangle$, a calibrated π pulse is applied on the qubit drive channel. Otherwise, the pulse is skipped.
4. **Wait**: allow any residual photons in the resonator to decay before starting the experiment.

Because the entire decision-to-pulse path runs inside the tProcessor, the latency between completing the readout window and firing the corrective pulse is determined solely by FPGA clock cycles. For the ZCU216 operating at a 384 MHz fabric clock, this latency is of order tens of nanoseconds — orders of magnitude faster than a software round-trip.

6.5 Latency Budget

Table 3 gives an estimated latency breakdown for the active reset sequence.

Table 3: Estimated latency contributions in the active reset loop.

Stage	Latency	Notes
Readout pulse duration	0.5 µs to 2 µs	Depends on κ
ADC integration window	0.5 µs to 2 µs	
FPGA decision (threshold)	∼10 ns	tProcessor cycles
Conditional branch	∼3 ns	1 tProc cycle
π pulse duration	50 ns to 500 ns	Depends on T_1
Total (excl. readout)	<1 µs	

The dominant latency is the readout integration window; the digital decision and conditional fire together contribute less than 100 ns, well within qubit coherence times of 10 µs to 100 µs for state-of-the-art fluxonium devices.

7 ML-Assisted State Discrimination

7.1 IQ Data and the Discrimination Problem

Each single-shot readout produces an averaged (I, Q) pair — a point in the complex plane. Repeated preparation and measurement of the qubit in the same state produces a cloud (cluster) of such points, whose centroid and spread depend on the readout power, resonator parameters, and integration time. Two such clusters, one for $|0\rangle$ and one for $|1\rangle$, must be distinguished reliably.

For ideal Gaussian clusters of equal covariance, the optimal linear classifier is the perpendicular bisector of the line segment joining the two centroids:

$$d(\mathbf{x}) = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1} \left(\mathbf{x} - \tfrac{1}{2}(\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1) \right) \underset{\hat{y}=0}{\overset{\hat{y}=1}{\gtrless}} 0, \tag{9}$$

where $\boldsymbol{\mu}_0, \boldsymbol{\mu}_1 \in \mathbb{R}^2$ are the class centroids and $\boldsymbol{\Sigma}$ is the common covariance matrix (Linear Discriminant Analysis, LDA). When the covariances differ, Quadratic Discriminant Analysis (QDA) uses the class-specific covariances and yields a curved decision boundary.

In practice, residual state preparation errors, qubit T_1 decay during the readout pulse, and higher-level leakage ($|2\rangle$, $|3\rangle$) distort the clusters from ideal Gaussians. These effects motivate the use of more expressive classifiers. The assignment fidelity is defined as

$$\mathcal{F} = \tfrac{1}{2}\big[P(\hat{s} = 0 \mid s = 0) + P(\hat{s} = 1 \mid s = 1)\big], \tag{10}$$

the average per-class accuracy across both prepared states.

7.2 Lightweight Classifiers for FPGA Deployment

For eventual on-FPGA deployment, the classifier must be computable using only fixed-point arithmetic and a small number of multiply-accumulate operations. The key constraints are:

- **Latency**: the decision must complete within one tProcessor cycle budget ($\lesssim$10 ns) to avoid extending the readout dead time.
- **Resource usage**: the classifier must fit in the available DSP slices and block RAM of the FPGA without displacing the readout and tProcessor logic.
- **Trainability**: the classifier must be trainable on a laptop from a few thousand calibration shots.

With these constraints in mind, three classifier families are considered. A **linear threshold** (LDA) reduces to a single dot product and a comparison — the minimum possible arithmetic, directly implementable in the tProcessor's `mathi` instruction. A **quadratic threshold** (QDA) evaluates a 2×2 matrix product (4 multiply-accumulates) and handles asymmetric cluster covariances. A **small MLP** with inputs (I, Q), one or two hidden layers of 4–64 neurons with ReLU activations, and a sigmoid output can be computed as a matrix-vector product and fits in $\lesssim 200$ 18-bit DSP slices on the ZCU216. The following sections report results from evaluating these classifiers offline on existing experimental data; FPGA deployment is discussed in Section 7.7.

7.3 Experimental Data

The dataset used throughout this chapter is `SSRO.h5`, collected on a superconducting qubit device at Qilimanjaro. It contains 2000 single-shot IQ measurements per prepared state ($|0\rangle$ and $|1\rangle$), accumulated on-FPGA by the rectangular integrator in `axis_readout_v2`. Each shot is a single (I, Q) pair — the time-averaged quadrature amplitudes over the readout window. The class centroids and separations extracted from this dataset are summarised in Table 4.

Table 4: IQ blob statistics extracted from `SSRO.h5`.

Quantity	$\|0\rangle$	$\|1\rangle$
Centroid μ_I (ADU)	-0.32	-2.10
Centroid μ_Q (ADU)	$+4.50$	$+6.31$
IQ separation (ADU)	2.53	
IQ correlation	-0.587	-0.664

The negative IQ correlation in both states indicates a rotated noise ellipse, consistent with a readout tone that is not perfectly aligned to the I axis. The data were split 80/20 into training and test sets (3200 / 800 shots) using stratified sampling.

7.4 Part I — Integrated IQ Discrimination

LDA and a representative MLP (hidden layers 32→16, ReLU) were trained on the integrated IQ data. Table 5 reports the assignment fidelity on the held-out test set.

Table 5: Assignment fidelity on integrated IQ data (test set, 800 shots).

Classifier	$\mathcal{F}$	$\|0\rangle$ acc.	$\|1\rangle$ acc.
LDA	0.906	0.940	0.873
MLP (32→16)	0.906	0.940	0.873

LDA and the MLP achieve identical fidelity. The $|0\rangle$ accuracy (94.0%) exceeds the $|1\rangle$ accuracy (87.3%), pointing at a systematic source of $|1\rangle$ misclassification. This asymmetry is the first diagnostic signature of T_1 relaxation during the readout window, which causes some $|1\rangle$ shots to produce an integrated IQ that falls in the $|0\rangle$ blob.

7.5 Part II — Architecture Search

To test whether the LDA–MLP parity in Part I is specific to the (32→16) architecture, a systematic grid search was performed over 20 MLP configurations spanning 1–3 hidden layers and 4–64 neurons per layer (17 to 6465 parameters), evaluated by 5-fold stratified cross-validation.

The result is striking: the fidelity landscape is entirely flat. No architecture — across three orders of magnitude in parameter count — improves on LDA. This is not a failure of the models; it is a statement about the data.

For two classes with Gaussian distributions $\mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$ and $\mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1)$, the Bayes-optimal boundary is a quadratic surface. When $\Sigma_0 \approx \Sigma_1$ — as approximately holds here, confirmed by the nearly identical IQ correlations in Table 4 — it collapses to the linear boundary found by LDA. An MLP can at best approximate this boundary; it cannot exceed the Bayes optimum.

Table 6: Architecture grid search results (5-fold CV fidelity). LDA baseline: 0.898 ± 0.013. No MLP architecture exceeds LDA.

Architecture	$\mathcal{F}$ (CV)	$\pm$ std	Δ vs. LDA
(4,)	0.897	0.012	−0.001
(8,)	0.897	0.012	−0.001
(16,)	0.898	0.012	−0.000
(32,)	0.898	0.012	−0.000
(64,)	0.897	0.012	−0.001
(8, 8)	0.896	0.012	−0.002
(16, 16)	0.897	0.012	−0.001
(32, 32)	0.897	0.012	−0.001
(64, 64)	0.897	0.012	−0.001
(16, 8)	0.897	0.012	−0.001
(32, 16)	0.897	0.012	−0.001
(16, 16, 8)	0.898	0.012	−0.000
(32, 32, 16)	0.897	0.013	−0.001
(64, 64, 32)	0.897	0.012	−0.002

The implication is fundamental: further fidelity improvement cannot come from a more expressive classifier on the same integrated IQ input. It must come from richer input features. The $|1\rangle$ accuracy deficit diagnosed in Part I identifies the mechanism directly: T_1 relaxation mid-shot.

7.6 Part III — Raw-Trace CNN

7.6.1 Motivation and the T1 Relaxation Problem

When a $|1\rangle$ qubit relaxes to $|0\rangle$ at some time t^* within the readout window, the FPGA integrator averages over both signal levels and produces an (I, Q) point that lands between the two blobs. Any classifier operating on this compressed input — LDA, QDA, or MLP — must guess; the information about the true prepared state has been discarded by the integration.

The raw ADC time trace, however, encodes t^* directly as a step in the signal. A temporal classifier trained on raw traces can in principle recover these shots by detecting the jump. Part III demonstrates this capability using physically-grounded synthetic traces, since raw decimated traces from the ZCU216/QICK setup are not yet available (acquiring them requires using `acquire_decimated` rather than the standard `acquire` call, and collecting a dedicated `SSRO_raw.h5` dataset with the board connected to a qubit chip).

7.6.2 Synthetic Trace Model

Each shot is modelled as a noisy cavity response:

$$\begin{pmatrix} i(t) \\ q(t) \end{pmatrix} = \frac{e(t)}{\bar{e}} \boldsymbol{\mu}_{s(t)} + \boldsymbol{\eta}(t), \qquad e(t) = 1 - e^{-t/\kappa}, \tag{11}$$

where $\bar{e} = \langle e(t) \rangle_t$ normalises the envelope so that $\langle (i(t), q(t)) \rangle_t = \boldsymbol{\mu}_{s(t)}$, matching the real blob centroids exactly. The noise $\boldsymbol{\eta}(t) \sim \mathcal{N}(\mathbf{0}, T\Sigma_s)$ is chosen so that the sample mean has covariance Σ_s, matching the real blob widths.

For $|1\rangle$ shots, qubit relaxation is modelled by drawing a relaxation time t^* from a geometric distribution with rate $1 - e^{-1/T_1}$. After t^* the signal switches from $\boldsymbol{\mu}_1$ to $\boldsymbol{\mu}_0$:

$$\boldsymbol{\mu}_{s(t)} = \begin{cases} \boldsymbol{\mu}_1 & t < t^* \\ \boldsymbol{\mu}_0 & t \geq t^* \end{cases}. \tag{12}$$

The simulation parameters are: readout window $T = 200$ samples ($\approx 1\,\mu$s at 200 MSps), cavity decay $\kappa^{-1} = 20$ samples (100 ns), and $T_1 = 600$ samples (3 µs). A self-consistency check confirmed that rectangular integration of the synthetic traces reproduces the real `SSRO.h5` blob statistics to within 1% (Table 7).

Table 7: Self-consistency check: synthetic integrated IQ vs. real `SSRO.h5`.

Quantity	Synthetic	Real
μ_I, $\|0\rangle$	-0.341	-0.324
μ_Q, $\|0\rangle$	$+4.521$	$+4.502$
σ_I, $\|0\rangle$	0.645	0.649
μ_I, $\|1\rangle$ (no relax)	-2.098	-2.095
μ_Q, $\|1\rangle$ (no relax)	$+6.336$	$+6.313$
σ_I, $\|1\rangle$ (no relax)	0.851	0.874

With $T_1 = 600$ samples ($\approx 3\times$ the readout window), the expected relaxation fraction among $|1\rangle$ shots is approximately 28–30%, corresponding to 566–603 relaxation events per 2000 shots. This is notably higher than the measured $|1\rangle$ error rate of approximately 12.75% observed in Table 5. The discrepancy indicates that the simulated T_1 is shorter than the actual qubit T_1 for this dataset, or equivalently that T_1 should be set higher (around 1200–1500 samples) to match the observed error rate. The relative CNN-versus-LDA comparison remains valid regardless of the exact T_1 value; the absolute fidelity numbers from Part III should be treated as simulation estimates rather than predictions for the real system.

7.6.3 CNN Architecture and Training

The `ReadoutCNN` model operates on raw traces of shape $(T, 2)$. Two `Conv1d` layers compress the time axis, global average pooling collapses it, and a small dense head produces the binary logit for $P(|1\rangle)$. The model has 953 parameters, sized for deployment via `hls4ml`.

Table 8: `ReadoutCNN` architecture summary (953 parameters).

Layer	Output shape	Parameters
Conv1d(2→8, k=5)	$(T-4,\ 8)$	88
ReLU + MaxPool1d(2)	$((T-4)/2,\ 8)$	—
Conv1d(8→16, k=5)	$(\cdot,\ 16)$	656
ReLU + GlobalAvgPool	$(16,)$	—
Linear(16→8)	$(8,)$	136
ReLU + Linear(8→1)	$(1,)$	73

The model was trained for 60 epochs using binary cross-entropy loss and the Adam optimiser. Test fidelity reached 0.994 by epoch 39 and remained stable thereafter.

7.6.4 Results

Table 9 compares LDA on integrated IQ against the CNN on raw traces, evaluated on the same held-out test set.

Table 9: CNN vs. LDA on synthetic raw traces. LDA operates on the time-averaged (I, Q); the CNN operates on the full trace $(T, 2)$. All values are from the held-out test set.

Classifier	$\mathcal{F}$	$\|0\rangle$ acc.	$\|1\rangle$ acc. (all)	$\|1\rangle$ acc. (relax. only)
LDA (integrated IQ)	0.826	0.875	0.778	0.443
1D CNN (raw trace)	0.994	0.995	0.993	0.984
CNN gain	+16.8%	+12.0%	+21.5%	+54.1%

The CNN achieves 98.4% accuracy on relaxation shots, compared to 44.3% for LDA — a gain of 54 percentage points on the shots that are entirely invisible to integration-based methods. The $P(|1\rangle)$ confidence as a function of relaxation time t^* rises monotonically: shots that relax late in the window (spending most of the trace in $|1\rangle$) are classified with near-certainty, while shots that relax early (spending most of the trace in $|0\rangle$) are correctly identified as ambiguous. This confirms the CNN is exploiting the temporal jump structure, not merely interpolating between blob positions.

These results should be understood as simulation estimates. The absolute fidelity numbers depend on the synthetic T_1 value (Section 7.6), which does not precisely match the real qubit's T_1. The qualitative conclusion — that a CNN on raw traces can recover T_1-relaxation shots that are irrecoverable from integrated IQ — is robust to this uncertainty.

7.7 FPGA Deployment Path

FPGA deployment of the trained classifiers was not achieved within the scope of this work. This section outlines the concrete steps required to close the loop from the trained models to real-time discrimination inside the QICK firmware.

The current readout pipeline — rectangular integrator followed by the tProcessor threshold register — already implements LDA implicitly. Formalising this as an explicit dot-product instruction (`mathi`) adds no hardware overhead. The CNN requires a separate data path: raw decimated traces from `acquire_decimated` are fed into an `hls4ml`-synthesised IP block that produces a binary decision within approximately 100 ns to 200 ns at 250 MHz, well within the tProcessor's feedback budget (Table 3).

The full deployment sequence is:

1. **Collect real raw traces**: connect the ZCU216 to the qubit chip and acquire `SSRO_raw.h5` using `acquire_decimated` (shape $(2, N_{\text{shots}}, T, 2)$). Set `readout_length` to at least $3/\kappa$ to capture the full cavity ringdown.

2. **Retrain and validate**: retrain the CNN on real traces. Recalibrate the synthetic T_1 parameter to match the measured $|1\rangle$ error rate ($\approx 12.75\%$), which requires $T_1 \approx$ 1200–1500 samples.

3. **Export to HLS**: export the trained PyTorch model to ONNX, then convert using `hls4ml` targeting the ZCU216 fabric (`xczu49dr-ffvf1760-2-e`), `io_stream`, and fixed-point precision `ap_fixed<16,6>`.

4. **Synthesise and integrate**: synthesise the HLS project in Vivado and integrate the resulting IP block into the QICK bitstream alongside the existing readout chain.

5. **Benchmark**: compare LDA-threshold, CNN-IP, and host-side discrimination on the same experimental data to quantify latency reduction and fidelity gain under realistic qubit conditions.

Based on the model's 953 parameters and typical `hls4ml` resource scaling for comparable architectures, the CNN IP block is expected to require on the order of 10 000 LUTs and tens of DSP slices — a small fraction of the ZCU216's available fabric — though this must be confirmed by synthesis.

8 Conclusions and Outlook

8.1 Summary

This thesis has documented the setup, characterisation, and initial evaluation of a QICK-based readout architecture for superconducting qubits on a Xilinx ZCU216 RFSoC platform at Qilimanjaro Quantum Tech, and has explored the design of a machine-learning-assisted state discrimination pipeline using existing experimental data.

The main contributions are:

1. **System integration**: The ZCU216 board, XM655 analogue front-end, and QICK firmware were brought up from scratch, including SSH access, PYNQ configuration, and a persistent Pyro4 remote-operation server that preserves calibration state across notebook sessions.

2. **Phase-coherent readout calibration**: A systematic calibration procedure was developed that measures the random DAC–ADC DDS phase offset at board power-on. The linear phase-vs-frequency model $\phi(f) = \phi_0 - 360°\tau f$ was validated, and the fitted `res_phase` value was incorporated into the standard experiment configuration.

3. **Readout data-path**: The trade-offs between decimated (waveform) and accumulated (averaged IQ) readout modes were characterised. The $1/\sqrt{N_{\text{reps}}}$ noise scaling of accumulated readout was confirmed, reducing single-shot noise from $\sim$1.9 to $\sim$0.14 ADU at 1000 repetitions. The resonator spectroscopy workflow was established for both single-qubit and multiplexed scenarios.

4. **Real-time feedback primitives**: The tProcessor-64 conditional branch (`condj`) was demonstrated in loopback as the core primitive for mid-circuit measurement. An active qubit reset sequence was designed, combining dispersive readout, thresholding, and a calibrated π pulse, with an estimated digital decision-to-fire latency below 100 ns.

5. **ML-assisted discrimination**: A complete offline discrimination pipeline was developed and evaluated on existing experimental SSRO data. On integrated IQ data, LDA achieves an assignment fidelity of 90.6% ($|0\rangle$: 94.0%, $|1\rangle$: 87.3%), and a systematic grid search over 20 MLP architectures confirms that no nonlinear classifier improves on this — a consequence of the near-Gaussian, linearly separable IQ blob structure. The $|1\rangle$ accuracy deficit is identified as the signature of T_1 relaxation mid-shot, which integration-based methods cannot recover. A lightweight 1D CNN (953 parameters) trained on physically-grounded synthetic raw traces achieves 99.4% overall fidelity and 98.4% accuracy on relaxation shots, compared to 44.3% for LDA on the same shots. FPGA deployment of the CNN was not achieved within the scope of this work; Section 7.7 outlines the concrete path toward it.

8.2 Outlook

Several natural extensions of this work follow directly from the results above:

CNN retraining on real raw traces. The CNN results in Chapter 7 are based on synthetic traces anchored to real IQ statistics. The first experimental priority is to collect a real `SSRO_raw.h5` dataset from the ZCU216 using `acquire_decimated`, retrain the CNN, and validate that the T_1-relaxation recovery generalises to real data. The synthetic T_1 parameter should be recalibrated to match the measured $|1\rangle$ error rate ($\approx 12.75\%$, implying $T_1 \approx 1200$–1500 samples rather than the 600 used in simulation) before the synthetic-to-real transfer is attempted.

On-FPGA classifier deployment. With real raw traces in hand, the CNN can be exported via `hls4ml` and synthesised as a fixed-point IP block in the QICK bitstream, as described in Section 7.7. This closes the active-reset feedback loop at full speed: the tProcessor receives a binary decision from the CNN IP block within 100 ns–200 ns, without a software round-trip.

Multi-qubit readout. The current architecture supports a single readout tone. Extending to multiplexed readout via the QICK PFB firmware would allow simultaneous readout of all qubits in Qilimanjaro's multi-qubit chips, which is a prerequisite for running quantum error-correction cycles.

Integration with Qilimanjaro's software stack. The QICK-based readout layer developed here should be integrated with Qilimanjaro's higher-level quantum programming interfaces (circuit compilation, calibration databases, experiment management) to provide a production-ready control stack.

Fluxonium-specific readout optimisation. The dispersive shift landscape for fluxonium qubits is two-dimensional in the (Φ_z, Φ_x) flux space. An ML-assisted optimisation of the readout circuit parameters — coupling capacitance, resonator frequency — to maximise the dimensionless cavity pull over a large flux region is a complementary research direction being pursued in a related BSc project within the team.

Bibliography

[1] Leandro Stefanazzi et al. QICK: Quantum instrumentation control kit for superconducting quantum computers. *Review of Scientific Instruments*, 93(4):044709, 2022. arXiv:2110.00557.

[2] Philip Krantz, Morten Kjaergaard, Fei Yan, Terry P. Orlando, Simon Gustavsson, and William D. Oliver. A quantum engineer's guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 2019.

[3] Jens Koch, Terri M. Yu, Jay Gambetta, A. A. Houck, D. I. Schuster, J. Majer, Alexandre Blais, M. H. Devoret, S. M. Girvin, and R. J. Schoelkopf. Charge-insensitive qubit design derived from the Cooper pair box. *Physical Review A*, 76(4):042319, 2007.

[4] Vladimir E. Manucharyan, Jens Koch, Leonid I. Glazman, and Michel H. Devoret. Fluxonium: Single Cooper-pair circuit free of charge offsets. *Science*, 326(5949):113–116, 2009.

[5] Alexandre Blais, Arne L. Grimsmo, S. M. Girvin, and Andreas Wallraff. Circuit quantum electrodynamics. *Reviews of Modern Physics*, 93(2):025005, 2021. arXiv:2005.12667.

[6] Alexandre Blais, Ren-Shou Huang, Andreas Wallraff, S. M. Girvin, and R. J. Schoelkopf. Cavity quantum electrodynamics for superconducting electrical circuits: An architecture for quantum computation. *Physical Review A*, 69(6):062320, 2004. arXiv:cond-mat/0402216.

[7] Jay Gambetta, Alexandre Blais, Maxime Boissonneault, Andrew A. Houck, D. I. Schuster, and S. M. Girvin. Protocols for optimal readout of qubits using a continuous quantum nondemolition measurement. *Physical Review A*, 76(1):012325, 2007.

A Key Code Listings

This appendix collects the most important Python/QICK code snippets used in this thesis. Full notebooks are available in the accompanying GitHub repository.

A.1 Phase Calibration Programme

```python
class PhaseCalProgram(AveragerProgram):
    """Sweeps frequency and measures DAC-ADC phase offset."""

    def initialize(self):
        cfg = self.cfg
        self.declare_gen(ch=cfg["res_ch"], nqz=1)
        self.declare_readout(ch=cfg["ro_ch"],
                             length=cfg["readout_length"],
                             freq=cfg["pulse_freq"],
                             gen_ch=cfg["res_ch"])
        freq_reg = self.freq2reg(cfg["pulse_freq"],
                                 gen_ch=cfg["res_ch"],
                                 ro_ch=cfg["ro_ch"])
        self.set_pulse_registers(ch=cfg["res_ch"],
                                 style="const",
                                 freq=freq_reg,
                                 phase=0,
                                 gain=cfg["pulse_gain"],
                                 length=cfg["length"])
        self.synci(200)

    def body(self):
        self.measure(pulse_ch=self.cfg["res_ch"],
                     adcs=[self.cfg["ro_ch"]],
                     adc_trig_offset=self.cfg["adc_trig_offset"],
                     wait=True,
                     syncdelay=self.us2cycles(self.cfg["relax_delay"]))
```

A.2 Fast Amplitude Sweep with `RAveragerProgram`

```python
class SweepProgram(RAveragerProgram):
    """Sweeps DAC gain inside the FPGA without Python overhead."""

    def initialize(self):
        cfg = self.cfg
        self.declare_gen(ch=cfg["res_ch"], nqz=1)
        self.declare_readout(ch=cfg["ro_ch"],
                             length=cfg["readout_length"],
                             freq=cfg["pulse_freq"],
                             gen_ch=cfg["res_ch"])
        freq_reg = self.freq2reg(cfg["pulse_freq"],
                                 gen_ch=cfg["res_ch"],
                                 ro_ch=cfg["ro_ch"])
        self.set_pulse_registers(ch=cfg["res_ch"],
                                 style="const",
                                 freq=freq_reg,
                                 phase=cfg["res_phase"],
                                 gain=cfg["start"],
                                 length=cfg["length"])
```

```
        self.r_rp   = self.ch_page(cfg["res_ch"])
        self.r_gain = self.sreg(cfg["res_ch"], "gain")
        self.synci(200)

    def body(self):
        self.measure(pulse_ch=self.cfg["res_ch"],
                     adcs=[self.cfg["ro_ch"]],
                     adc_trig_offset=self.cfg["adc_trig_offset"],
                     wait=True,
                     syncdelay=self.us2cycles(self.cfg["relax_delay"]))

    def update(self):
        self.mathi(self.r_rp, self.r_gain, self.r_gain,
                   '+', self.cfg["step"])
```

A.3 Active Reset Sequence (Pseudocode)

Algorithm 1 Active qubit reset

Require: Calibrated π-pulse amplitude A_π and readout threshold θ

Trigger ADC readout window
Play readout tone on resonator channel
wait for readout window to close
$b \leftarrow \text{FPGA\_threshold}(I + jQ, \theta)$ ▷ $b = 1$ if qubit in $|1\rangle$
regwi $r_b \leftarrow b$
condj $r_b < 1$ **goto** SKIP_PI
Play π pulse on qubit drive channel
label SKIP_PI
synci t_{relax} ▷ Allow cavity photons to decay